

TD VAT 2

Validation par simulation et viabilité

La simulation consiste à tracer le fonctionnement attendu de l'ordonnancement d'un système. Elle ne permet de représenter qu'un comportement unique : pour une période donnée pour chaque tâche, une durée donnée, une date de réveil donnée. Cependant, nous avons vu que la durée d'exécution en-ligne était variable, qu'il existait des tâches sporadiques dont la période représente en fait le délai minimal entre deux activations successives, et que certains systèmes sont concrets (dates de réveil connues) alors que d'autres sont non-concrets (dates de réveil inconnues).

La simulation permet de valider un système si et seulement si elle représente le comportement pire cas du système. Cela n'est vrai que si la simulation est **viable** dans le **contexte** pour les paramètres qui ne sont pas fixés.

Contexte : hypothèses s'appliquant au système étudié. Par exemple : « ordonnanceur en-ligne sur un système monoprocesseur, tâches périodiques simultanées indépendantes, et à échéances sur requête. »

Quelques éléments liés au contexte :

Processeur : monoprocesseur, multiprocesseur global, multiprocesseur partitionné, réparti

Ordonnanceur : en-ligne ou hors-ligne

Tâches : concrètes ou non

Tâches concrètes : simultanées ou non

Activations des tâches : périodiques, sporadiques, apériodiques, reliées

Interdépendances des tâches : indépendantes, avec sections critiques (ou portions non préemptibles), contraintes de précedence

Structure interne des tâches : avec ou sans suspension

Viabilité :

Un test d'ordonnançabilité est **C-viable** dans un contexte si réduire la durée d'exécution ne peut augmenter de temps de réponse.

Il est **T-viable** dans un contexte si augmenter une période ne peut augmenter un temps de réponse.

Il est **r-viable** si augmenter une date de réveil ne peut augmenter un temps de réponse.

Exercice 1 :

Soit un système de tâches indépendantes, périodiques, ordonné en-ligne par un ordonnanceur à priorités fixes aux tâches sur un processeur, caractérisé par :

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i	Priorité Π_i
τ_1	0	8	30	30	1
τ_2	0	6	20	20	2
τ_3	0	4	15	15	3

L'**utilisation processeur** $U = \sum_{i=1}^n \frac{C_i}{T_i}$ (aussi appelée charge processeur) d'un système fournit une condition nécessaire triviale d'ordonnançabilité. Il faut en effet que $U \leq k$ pour qu'un système ait une

chance d'être ordonnançable sur un système de k processeurs. Si $U > 1$, le système est trivialement non-ordonnançable sur 1 processeur.

- Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.
- Tracer la simulation de l'ordonnancement du système, puis vérifier que la séquence produite est la même que celle produite par SimSo Web (<http://projects.laas.fr/simso/simso-web/#/configuration>).
- Quel est le pire temps de réponse observé de chaque tâche ? Chaque tâche respecte-t-elle toutes ses échéances ?

Lorsque les tâches sont indépendantes, préemptibles, et ordonnancées sur un processeur, la **simulation est C-viable et T-viable**.

De plus, si le système est non concret, la simulation du système de tâches en considérant des réveils simultanés est **r-viable**.

- Le système est-il ordonnançable par Rate Monotonic ?
- En supposant que les dates de réveil soient inconnues, le système est-il ordonnançable ?
- Tracer la simulation de l'ordonnancement du système en supposant cette fois un ordonnanceur Earliest Deadline First, puis vérifier que la séquence produite est la même que celle produite par SimSo Web.
- Quel est le pire temps de réponse de chaque tâche ?
- Le système est-il ordonnançable par EDF ?

Exercice 2 :

Considérons un système de tâches différées, dont les paramètres sont donnés ci-après :

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i	Priorité Π_i
τ_1	8	8	30	30	1
τ_2	4	6	20	20	2
τ_3	0	6	15	15	3

- Calculer l'utilisation processeur (aussi appelée charge processeur) U du système, et conclure.

Si le système est ordonnançable, en monoprocesseur, avec $D_i \leq T_i$, la première fenêtre de taille $H = \text{PPCM}(T_i)$ contenant exactement $H(1-U)$ temps creux donne le cycle de l'ordonnancement. Ceci arrive au plus tard à la date $\max(r_i) + 2H$.

- Tracer la simulation de l'ordonnancement du système, puis vérifier que la séquence produite est la même que celle produite par SimSo Web.
- Tracer la simulation en supposant que les tâches sont simultanées, et conclure.

Pour une tâche de priorité Π_i , le cas le plus défavorable survient lorsqu'elle subit la pire interférence de toutes les tâches de priorité plus élevée. L'**instant critique** d'une tâche survient lorsqu'elle est activée simultanément avec toutes les tâches plus prioritaires, car cela maximise l'interférence de celles-ci.

- En supposant les dates de réveil données dans la table, mais à supposer cette fois que les tâches sont sporadiques, que conclure ?

Exercice 3 :

Nous considérons un système de tâches pour lesquelles τ_3 précède τ_1 .

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i	Priorité Π_i
τ_1	0	8	12	12	3
τ_2	1	2	3	12	2
τ_3	0	2	12	12	1

- Tracer la simulation de l'ordonnancement produit.
- Supposer maintenant que $C_3=1$, tracer la simulation.
- Que conclure sur la viabilité de la simulation lorsqu'il existe des contraintes de précédence ?

Exercice 4 :

Nous considérons un système de tâches pour lesquelles τ_1 et τ_2 sont en exclusion mutuelle pendant toute leur durée. Nous supposons une affectation de priorités Deadline Monotonic (priorité inversement proportionnelle au délai critique).

	Date de réveil r_i	WCET C_i	Délai critique D_i	Période T_i
τ_1	0	6	16	16
τ_2	0	2	6	8
τ_3	0	6	15	16

- Tracer la simulation de ce système.
- Tracer une simulation lorsque $C_3=5$
- Que conclure sur la viabilité de la simulation lorsqu'il existe des portions de tâches non préemptibles ?